

Documentación Técnica del Proyecto: local_prompt_tarea

Este documento describe la arquitectura, la base de datos, la lógica de negocio, los hooks y el flujo de trabajo del plugin local de Moodle `local_prompt_tarea`.

1. Introducción y Arquitectura General

1.1 Propósito del Plugin

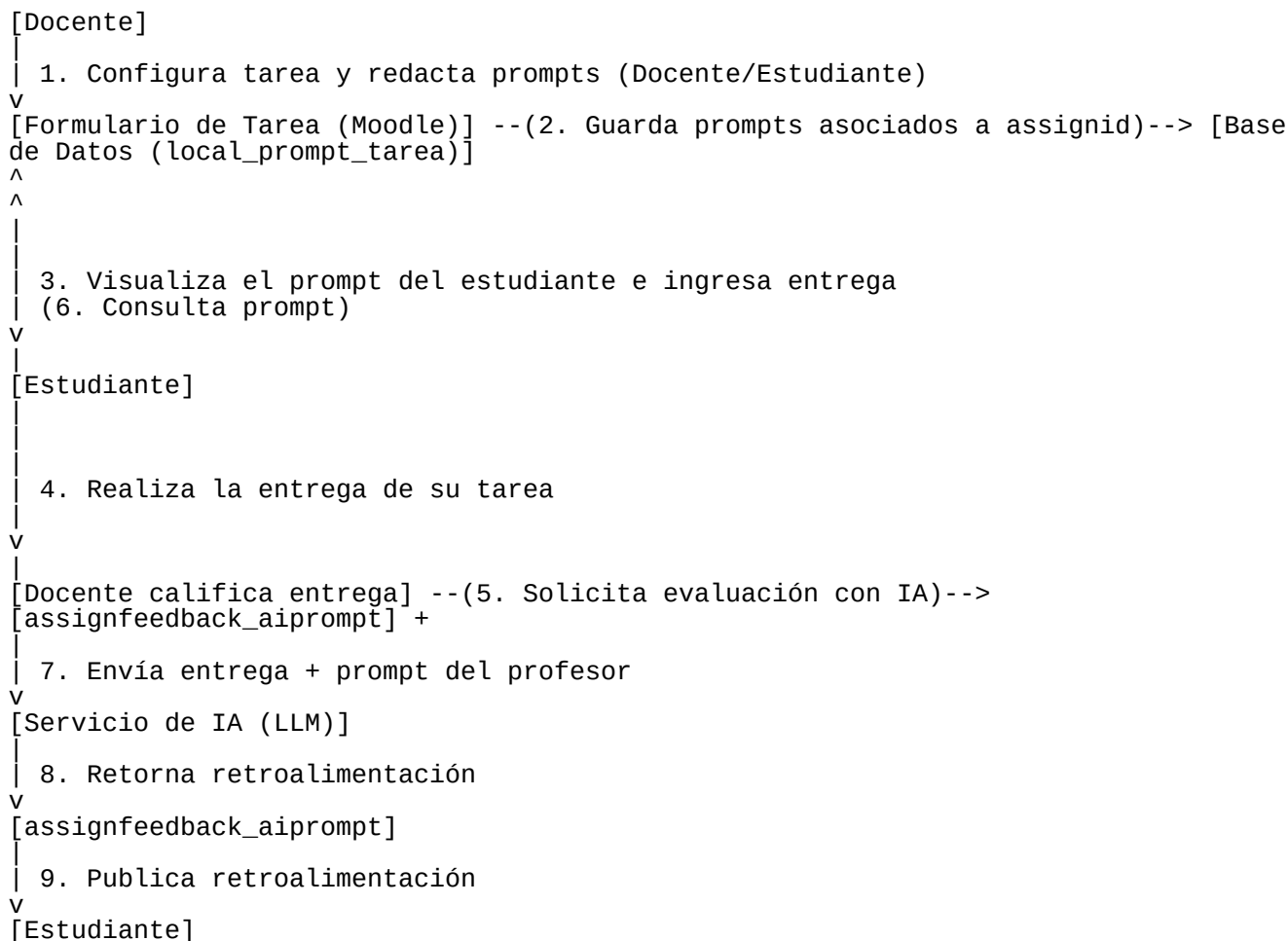
El plugin `local_prompt_tarea` es una extensión del tipo **local** para Moodle diseñada para ejecutarse en la versión **5.0+** con **PHP 7.4+**. Su objetivo principal es permitir a los profesores asociar dos tipos de prompts de Inteligencia Artificial a cada actividad de tipo Tarea (assign) en Moodle:

- 1. Prompt de evaluación (Profesor):** Instrucciones o rúbricas personalizadas que una IA debe seguir para evaluar las entregas de los estudiantes.
- 2. Prompt para el estudiante:** Instrucciones de apoyo, consejos o directrices generadas por el docente para que el alumno las vea y las tenga en cuenta antes de realizar su entrega.

Este plugin actúa como un **proveedor de datos** clave dentro del ecosistema de Moodle, permitiendo que otros plugins (como el plugin de retroalimentación de tareas `assignfeedback_aiprompt`) lean estos campos para realizar la integración con modelos de lenguaje a gran escala (LLMs) y generar retroalimentaciones automáticas basadas en la configuración específica de cada tarea.

1.2 Flujo de Trabajo

A continuación se presenta un diagrama de secuencia que detalla la interacción entre el docente, el estudiante, la base de datos del plugin local y el módulo de feedback con IA:



2. Requisitos del Entorno e Instalación

2.1 Requisitos del Sistema

Para garantizar el correcto funcionamiento del plugin, el entorno de Moodle debe cumplir los siguientes requisitos mínimos:

Componente	Versión Mínima	Nota
Moodle	5.0 (Build 2024042200)	Compatible con la API de formularios y base de datos moderna.
PHP	7.4+	Soporte completo para tipos de datos e interacciones del core.
Base de Datos	MariaDB 10.4+, MySQL 5.7+ o PostgreSQL 12+	Soporte para el esquema XMLDB estándar.

2.2 Proceso de Instalación

Obtención y copia de archivos: Descarga o clona el código fuente del plugin y colócalo en el directorio de plugins locales de Moodle en la siguiente ruta: `bash {moodle_root}/local/prompt_tarea/`

Instalación desde la interfaz de Moodle:

- Inicia sesión en tu plataforma Moodle con una cuenta de Administrador.
- Dirígete a: **Administración del sitio** -> **Notificaciones**.
- Moodle detectará la presencia del nuevo plugin local y te presentará una pantalla de actualización.
- Haz clic en **Actualizar base de datos de Moodle ahora**.

Instalación mediante CLI (Línea de comandos): Si prefieres realizar la actualización desde la terminal de tu servidor, ejecuta el siguiente comando desde la raíz de Moodle: `bash php admin/cli/upgrade.php`

Una vez finalizado el proceso de instalación o actualización de la base de datos, la tabla `local_prompt_tarea` estará creada en el esquema.

3. Esquema de Base de Datos

El plugin almacena la información de los prompts en una tabla dedicada en la base de datos de Moodle para evitar la sobrecarga de la tabla principal de tareas (`mdl_assign`).

3.1 Estructura de la Tabla: `local_prompt_tarea`

La tabla física en la base de datos de Moodle se crea con el prefijo estándar de la plataforma (por ejemplo, `mdl_local_prompt_tarea`). La definición estructural es la siguiente:

Campo	Tipo de Datos	Nulo	Por Defecto	Descripción
id	BIGINT(10)	NO	Autoincremental	Clave primaria única para el registro del plugin.
assignid	BIGINT(10)	NO	Ninguno	Clave foránea externa que apunta al ID de la tarea (<code>mdl_assign.id</code>).

Campo	Tipo de Datos	Nulo	Por Defecto	Descripción
prompt	TEXT	SÍ	NULL	Texto del prompt configurado por el profesor para la evaluación por IA.
prompt_estudiante	TEXT	SÍ	NULL	Texto del prompt de ayuda visible para los estudiantes.
timecreated	BIGINT(10)	NO	0	Timestamp Unix que representa la fecha y hora de creación del registro.
timemodified	BIGINT(10)	NO	0	Timestamp Unix que representa la fecha y hora de la última modificación.

3.2 Índices y Restricciones

- **Clave Primaria (PRIMARY KEY):** Definida en el campo id.
- **Clave Foránea (FOREIGN KEY):** El campo assignid está enlazado a la tabla de tareas assign en su campo id.
- **Restricción Única (UNIQUE INDEX):** El campo assignid cuenta con un índice único (assignid_idx). Esto restringe el comportamiento del plugin para que cada tarea de Moodle posea, a lo sumo, una única configuración de prompts (relación de cardinalidad 1 a 1).

4. Detalle de Implementación Técnica (Hooks en lib.php)

El núcleo operativo del plugin se implementa a través de tres hooks estándar de Moodle definidos en lib.php. Estos hooks permiten interceptar el ciclo de vida del formulario de las tareas y realizar operaciones CRUD seguras.

4.1 Incorporación de Campos al Formulario de Moodle

La función `local_prompt_tarea_coursemodule_standard_elements` es responsable de inyectar los campos de configuración de prompts dentro del formulario nativo de edición de Moodle (`moodleform`).

```
function local_prompt_tarea_coursemodule_standard_elements($formwrapper, $mform) {
    global $DB;

    // Solo aplicar a módulos de tipo 'assign' (Tareas)
    if ($formwrapper->get_current()->modulename != 'assign') {
        return;
    }

    // 1. Sección y campo para el prompt del Docente
    $mform->addElement('header', 'promptheader', get_string('promptheader',
        'local_prompt_tarea'));
    $mform->addElement('textarea', 'prompt_tarea_text',
        get_string('promptlabel', 'local_prompt_tarea'),
        ['wrap="virtual" rows="10" cols="50"']);
    $mform->setType('prompt_tarea_text', PARAM_TEXT);
    $mform->addHelpButton('prompt_tarea_text', 'promptlabel', 'local_prompt_tarea');

    // 2. Sección y campo para el prompt del Estudiante
    $mform->addElement('header', 'promptestudianteheader',
        get_string('promptestudianteheader', 'local_prompt_tarea'));
    $mform->addElement('textarea', 'prompt_estudiante_text',
        get_string('promptestudiantelabel', 'local_prompt_tarea'),
```

```
'wrap="virtual" rows="10" cols="50"');
$mform->setType('prompt_estudiante_text', PARAM_TEXT);
$mform->addHelpButton('prompt_estudiante_text', 'promptestudiantelabel',
'local_prompt_tarea');

// 3. Carga segura de datos preexistentes si se edita la tarea
if (isset($formwrapper->get_current()->instance) &&
$formwrapper->get_current()->instance) {
try {
$assignid = $formwrapper->get_current()->instance;
$dbman = $DB->get_manager();
$table = new xmldb_table('local_prompt_tarea');

// Verificación defensiva contra fallos de base de datos durante actualizaciones
if ($dbman->table_exists($table)) {
$record = $DB->get_record('local_prompt_tarea', ['assignid' => $assignid]);
if ($record) {
$mform->setDefault('prompt_tarea_text', $record->prompt);
$mform->setDefault('prompt_estudiante_text', $record->prompt_estudiante);
}
} catch (Exception $e) {
debugging('Error loading prompt data: ' . $e->getMessage(), DEBUG_DEVELOPER);
}
}
}
```

- **Validación del módulo:** Se evalúa que el formulario pertenezca a un módulo assign. Si no es el caso, finaliza de inmediato.
- **Elementos del formulario:** Añade encabezados independientes (header) y entradas de texto multilinea (textarea). Se les asigna el tipo de entrada de texto seguro (PARAM_TEXT) y se añade un botón de ayuda contextual (addHelpButton).
- **Seguridad y resiliencia:** Utiliza \$dbman->table_exists dentro de un bloque try-catch para evitar fallos catastróficos en caso de que el plugin se intente configurar durante un proceso de migración de base de datos incompleto.

4.2 Almacenamiento Seguro de Datos

El almacenamiento de los prompts se gestiona mediante la intercepción del hook posterior a la acción de guardado de un módulo de curso (coursemodule_edit_post_actions).

```
function local_prompt_tarea_coursemodule_edit_post_actions($data, $course) {
global $DB;

if ($data->modulename != 'assign') {
return $data;
}

$dbman = $DB->get_manager();
$table = new xmldb_table('local_prompt_tarea');

if (!$dbman->table_exists($table)) {
debugging('Table local_prompt_tarea does not exist', DEBUG_DEVELOPER);
return $data;
}

if (isset($data->prompt_tarea_text) || isset($data->prompt_estudiante_text)) {
$time = time();

try {
// Comprobación de existencia del registro asociado a la tarea
$record = $DB->get_record('local_prompt_tarea', ['assignid' => $data->instance]);

if ($record) {
// Operación: Actualización (Update)
$record->prompt = $data->prompt_tarea_text ?? $record->prompt;
$record->prompt_estudiante = $data->prompt_estudiante_text ??
$record->prompt_estudiante;
$record->timemodified = $time;
}
```

```

$DB->update_record('local_prompt_tarea', $record);
} else {
// Operación: Creación (Insert)
$record = new stdClass();
$record->assignid = $data->instance;
$record->prompt = $data->prompt_tarea_text ?? '';
$record->prompt_estudiante = $data->prompt_estudiante_text ?? '';
$record->timecreated = $time;
$record->timemodified = $time;
$DB->insert_record('local_prompt_tarea', $record);
}
} catch (Exception $e) {
debugging('Error saving prompt: ' . $e->getMessage(), DEBUG_DEVELOPER);
}
}

return $data;
}

```

- **Estrategia Upsert:** Al guardar, la función comprueba la existencia de un registro para ese ID de tarea. Si ya existe un registro de prompt anterior, realiza un `update_record` modificando solo los campos requeridos y guardando la fecha de última modificación (`timemodified`). De lo contrario, inicializa un objeto PHP genérico (`stdClass`) y realiza un `insert_record`.

4.3 Eliminación en Cascada

Para evitar la acumulación de datos huérfanos e inútiles en la base de datos, el plugin implementa un hook que detecta cuándo una tarea ha sido eliminada por completo de Moodle.

```

function local_prompt_tarea_pre_course_module_delete($cm) {
global $DB;

if ($cm->modname == 'assign') {
$dbman = $DB->get_manager();
$table = new xmldb_table('local_prompt_tarea');

if ($dbman->table_exists($table)) {
try {
// Eliminación en cascada de los registros correspondientes
$DB->delete_records('local_prompt_tarea', ['assignid' => $cm->instance]);
} catch (Exception $e) {
debugging('Error deleting prompt: ' . $e->getMessage(), DEBUG_DEVELOPER);
}
}
}
}
}

```

5. Ciclo de Vida y Migración de Base de Datos

En el desarrollo de plugins para Moodle, las modificaciones de la base de datos a lo largo del ciclo de vida del software deben ser seguras y no destructivas. El plugin implementa esta estrategia en el archivo `db/upgrade.php` mediante la función `xmldb_local_prompt_tarea_upgrade`.

5.1 Adición Dinámica del campo `prompt_estudiante` (Versión 2025053100)

En su primera iteración, la tabla `local_prompt_tarea` únicamente almacenaba el `prompt` de evaluación del profesor. Al incorporar la característica del `prompt` para estudiantes en la actualización con versión 2025053100, se ejecutó el siguiente bloque de actualización:

```

function xmldb_local_prompt_tarea_upgrade($oldversion) {
global $DB;
$dbman = $DB->get_manager();

```

```

if ($oldversion < 2025053100) {
// 1. Instanciar la definición de la tabla
$table = new xmldb_table('local_prompt_tarea');

// 2. Definir el nuevo campo prompt_estudiante
$field = new xmldb_field('prompt_estudiante', XMLDB_TYPE_TEXT, null, null, null,
null, null, 'prompt');

// 3. Agregar el campo de forma segura si no existía previamente
if (!$dbman->field_exists($table, $field)) {
$dbman->add_field($table, $field);
}

// 4. Actualizar el savepoint en Moodle para registrar que la base de datos está al día
upgrade_plugin_savepoint(true, 2025053100, 'local', 'prompt_tarea');
}

return true;
}

```

- **xmldb_field:** El constructor de campo define el tipo como XMLDB_TYPE_TEXT e indica que debe posicionarse después del campo prompt.
 - **upgrade_plugin_savepoint:** Registra en la base de datos de configuración de Moodle que el plugin local ha completado correctamente su actualización a la versión 2025053100.
-

6. Internacionalización e Idioma (i18n)

Para soportar de manera nativa la visualización de textos en múltiples idiomas, las cadenas de caracteres se externalizan en archivos ubicados en la carpeta lang/.

El archivo de traducción se encuentra en local_prompt_tarea.php. A continuación se muestra su estructura y propósito de traducción:

```

<?php
$string['pluginname'] = 'Prompt para Tareas';
$string['promptheader'] = 'Prompt para asistencia a la evaluación';
$string['promptlabel'] = 'Agregar Prompt';
$string['promptlabel_help'] = 'Ingresa el prompt que desea asociar con esta tarea. Este texto se guardará en la base de datos.';
$string['promptestudianteheader'] = 'Prompt para retroalimentación de estudiantes';
$string['promptestudiantelabel'] = 'Agregar Prompt';
$string['promptestudiantelabel_help'] = 'Ingresa el prompt que será visible para el estudiante. Este texto se guardará en la base de datos.';
$string['privacy:metadata'] = 'El plugin Prompt para Tareas no almacena datos personales.';

```

- **privacy:metadata:** Esta cadena especial es utilizada de forma automática por el API de Privacidad de Moodle (Privacy API) para responder a las solicitudes de exportación y eliminación de datos bajo normativas internacionales como GDPR.
-

7. Seguridad y Privacidad (GDPR)

En entornos educativos institucionales, la privacidad de los datos es un aspecto crítico. El plugin local_prompt_tarea ha sido diseñado bajo los principios de **Privacidad por Diseño (Privacy by Design)**:

1. **Sin datos de carácter personal (PII):** El plugin no almacena nombres, correos electrónicos, datos de sesión ni identificadores de los estudiantes de Moodle.
 2. **Sin datos de calificaciones:** La tabla local_prompt_tarea solo guarda metadatos estructurales de la tarea (assignid) y las instrucciones escritas exclusivamente por el profesor de la asignatura.
 3. **Cumplimiento del API de Privacidad:** El plugin responde directamente al sistema de privacidad de Moodle declarando a través de \$string['privacy:metadata'] que no recolecta ni procesa datos personales de los usuarios.
-

8. Guía de Usuario para Docentes

El uso práctico del plugin es directo y se integra de forma transparente en la interfaz normal de edición de Moodle.

8.1 Configuración de Prompts en una Tarea

Cuando un docente crea una nueva tarea o edita una ya existente (dentro de los ajustes de una actividad Tarea en Moodle), notará la presencia de dos nuevas secciones en el formulario:

1. Sección: "Prompt para asistencia a la evaluación"

- **Uso:** En este espacio, el docente redacta las directivas de evaluación de Inteligencia Artificial.
- **Ejemplo de Prompt:** > *"Evalúa la siguiente entrega basándote en una escala de 1 a 5. Verifica que contenga una introducción clara, un análisis fundamentado y conclusiones coherentes. Proporciona una retroalimentación constructiva señalando áreas de mejora."*

2. Sección: "Prompt para retroalimentación de estudiantes"

- **Uso:** Este campo es el prompt complementario opcional que los estudiantes pueden visualizar en la página de descripción de la tarea para guiarlos antes de realizar la entrega.
- **Ejemplo de Prompt:** > *"Recuerda estructurar tu trabajo con un enfoque analítico. Utiliza referencias bibliográficas y cuida la ortografía. Este prompt te ayudará a orientar tu desarrollo."*

8.2 Buenas Prácticas para la Redacción de Prompts

Para maximizar la efectividad de la retroalimentación automática generada posteriormente por IA, se recomienda al docente estructurar el prompt de evaluación de la siguiente manera: 1. **Establecer el Rol:** Definir claramente el papel del evaluador de IA (ej. *"Actúa como un profesor universitario experto en sociología..."*). 2. **Criterios de Evaluación:** Listar con viñetas o números los aspectos que la IA debe verificar (ej. estructura, formato, calidad del análisis). 3. **Tono y Formato:** Indicar el tono esperado para la retroalimentación (ej. constructivo, motivador, formal) y el formato del texto de salida (ej. usar viñetas, secciones).